

Sessions 14 & 15: Detailed Explanation and Examples

1. Introduction to Selenium (using Eclipse IDE)

Selenium is one of the most popular open-source frameworks for automating web applications. It allows developers to automate the interactions with a web browser to simulate user behavior such as clicking, typing, selecting options, etc. Selenium supports various programming languages like Java, Python, C#, and JavaScript. In this context, we will focus on using **Java** with **Eclipse IDE**.

Why Selenium?

- **Cross-browser compatibility:** Selenium supports multiple browsers like Chrome, Firefox, Safari, etc.
- **Multi-language support:** Selenium can be used with different programming languages.
- **WebDriver:** It provides a robust and efficient tool for web browser automation.

Eclipse IDE:

- Eclipse is an Integrated Development Environment (IDE) that helps in developing Java applications, including web automation projects using Selenium.
- **Steps to set up Selenium in Eclipse:**
 1. **Install Eclipse IDE:** Download and install Eclipse IDE from the official website.
 2. **Install Java:** Ensure Java Development Kit (JDK) is installed on your system.
 3. **Add Selenium libraries:** Download the Selenium WebDriver libraries and add them to your Eclipse project's build path.

Example of Setup in Eclipse:

1. Open Eclipse IDE, create a new **Java Project**.
2. Right-click on the project, choose **Build Path > Configure Build Path > Libraries > Add External JARs**.
3. Add the **Selenium WebDriver JARs** you downloaded.
4. Create a Java class file where you will write your Selenium scripts.

2. Load WebDriver

The **WebDriver** is the core component in Selenium that controls the browser. It communicates with the browser and simulates user actions. To load the WebDriver, you need to instantiate the browser driver (e.g., ChromeDriver, FirefoxDriver) and set the path for the WebDriver executable.

Steps to Load WebDriver:

1. First, download the WebDriver executable for the browser you are automating (e.g., **ChromeDriver** for Google Chrome).
2. You need to set the path to the driver executable in your script.
3. Instantiate the WebDriver object and open the browser.

Example:

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver;

public class SeleniumExample {
    public static void main(String[] args) {
        // Set path to the WebDriver executable
        System.setProperty("webdriver.chrome.driver",
"path_to_chromedriver.exe");

        // Create an instance of ChromeDriver
        WebDriver driver = new ChromeDriver();

        // Open a website
        driver.get("https://www.example.com");

        // Perform actions (next steps)

        // Close the browser
        driver.quit();
    }
}
```

In this example, `ChromeDriver` is used to open the Chrome browser and navigate to ["www.example.com"](https://www.example.com). The `System.setProperty()` method points to the location of the `WebDriver` executable.

3. Create Selenium Commands: Locators

Locators in Selenium are used to find elements on a webpage. Selenium supports different types of locators for identifying HTML elements:

1. By ID

The **ID locator** is one of the most common and efficient locators. It finds an element by its `id` attribute. Since `id` values are unique on a webpage, it's a reliable locator.

Example:

```
driver.findElement(By.id("username")).sendKeys("testuser");
```

In this example, the `findElement` method locates the element with the `id="username"` and enters the text `testuser` into the input field.

2. By Name

The **name locator** is used when the `name` attribute is specified for an element. It works similarly to the ID locator.

Example:

```
driver.findElement(By.name("password")).sendKeys("password123");
```

Here, the script finds the element with the `name="password"` and inputs the text `password123`.

3. By Class Name

The **class name locator** finds an element by its `class` attribute. It's commonly used for finding multiple elements with the same class.

Example:

```
driver.findElement(By.className("login-button")).click();
```

This example clicks on the button with the class `login-button`.

4. By Tag Name

The **tag name locator** finds an element by its HTML tag name. This can be used to find elements like `<input>`, `<div>`, `<a>`, etc.

Example:

```
driver.findElement(By.tagName("a")).click();
```

Here, the script clicks on the first anchor (`<a>`) tag it finds on the page.

5. By XPath

XPath is a powerful locator that allows you to find elements based on their XML path, which can include attributes, text content, etc. It can handle complex queries.

Example:

```
driver.findElement(By.xpath("//input[@name='username']")).sendKeys("user123");
```

In this example, the XPath `//input[@name='username']` locates an input element where the name attribute is `username`.

4. Add Interactions

Selenium allows us to interact with elements on a webpage, simulating user actions. Below are common interactions that can be performed:

1. Text Box Interaction (Send Keys)

You can interact with text fields by entering text using the `sendKeys()` method.

Example:

```
driver.findElement(By.id("search-box")).sendKeys("Selenium WebDriver");
```

This enters the text `Selenium WebDriver` into the input field with `id="search-box"`.

2. Radio Button Selection

To select a **radio button**, you can use the `click()` method if the radio button is not selected already.

Example:

```
driver.findElement(By.id("gender-male")).click();
```

In this example, the radio button with `id="gender-male"` is clicked.

3. Check Box Selection

To select a **checkbox**, the `click()` method is used. If the checkbox is already selected, clicking it will unselect it.

Example:

```
driver.findElement(By.id("subscribe-checkbox")).click();
```

This clicks the checkbox with `id="subscribe-checkbox"`, checking it.

4. Drop-down Item Selection

For **drop-down lists**, Selenium provides a `Select` class to interact with dropdowns, allowing selection by visible text, index, or value.

Example:

```
import org.openqa.selenium.support.ui.Select;

Select dropdown = new Select(driver.findElement(By.id("country-dropdown")));
dropdown.selectByVisibleText("India");
```

Here, the script finds the drop-down with `id="country-dropdown"` and selects the option with the text `India`.

5. Keyboard Actions

You can simulate keyboard actions using the `Actions` class or the `sendKeys()` method. For more advanced actions like pressing the Enter key, Shift key, etc., use the `Actions` class.

Example:

```
driver.findElement(By.id("search-box")).sendKeys(Keys.ENTER);
```

In this example, the script simulates pressing the `Enter` key in the search box.

6. Mouse Actions

You can perform mouse interactions using the `Actions` class. This includes actions like clicking, right-clicking, hovering, and dragging.

Example (Mouse Hover):

```
import org.openqa.selenium.interactions.Actions;

Actions actions = new Actions(driver);
actions.moveToElement(driver.findElement(By.id("menu-item"))).perform();
```

This example simulates hovering over an element with `id="menu-item"`, which could trigger a dropdown menu.

7. Multi-Select Interaction

To interact with a **multi-select list**, Selenium provides methods to select multiple options in a `Select` element.

Example:

```
Select multiSelect = new Select(driver.findElement(By.id("languages")));  
multiSelect.selectByVisibleText("Java");  
multiSelect.selectByVisibleText("Python");
```

Here, the script selects multiple options (Java and Python) from a multi-select dropdown with `id="languages"`.

Summary:

- **Selenium** allows for automating web applications by interacting with various browser elements like text boxes, radio buttons, checkboxes, and dropdowns.
- The **WebDriver** object is used to load and control the browser.
- **Locators** in Selenium such as ID, name, class name, tag name, and XPath are used to find elements on a webpage.
- **Interactions** like entering text, selecting options, clicking buttons, and performing mouse and keyboard actions are crucial for simulating user behavior.

By mastering these concepts, you can automate a variety of tasks in web applications and test their functionalities effectively using Selenium.